

Reviewer Pack — Invariant Polynomial Generation Complexity in Determinant vs...

Entry 934a387d12d9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 16:16:27 UTC

Invariant Polynomial Generation Complexity in Determinant vs Permanent Orbits

Entry ID: 934a387d12d9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 16:16:27 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Algebraic Groups **Field B** (complexity object): Complexity of Polynomial Functions

Statement:

The minimal number of algebraically independent invariant polynomials required to generate the coordinate ring of the determinant's orbit under the action of $GL(n) \times GL(n) \times GL(n)$ is strictly less than that required for the permanent's orbit for all $n \geq 2$.

Rationale (proposer's reasoning):

The determinant's orbit is known to have a simpler structure in terms of invariant polynomials, which could reflect its lower computational complexity. The permanent's orbit may require more invariant polynomials due to its higher complexity, suggesting a deeper algebraic structure that is harder to compute.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ece581d94b3ca032

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    result = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_add(A, B):
    m = len(A)
    n = len(A[0])
    result = [[A[i][j] + B[i][j] for j in range(n)] for i in range(m)]
    return result

def matrix_subtract(A, B):
    m = len(A)
    n = len(A[0])
    result = [[A[i][j] - B[i][j] for j in range(n)] for i in range(m)]
    return result

def transpose(A):
    m = len(A)
    n = len(A[0])
    result = [[A[j][i] for j in range(m)] for i in range(n)]
    return result
```

```

def determinant(A):
    if len(A) == 1:
        return A[0][0]
    elif len(A) == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    else:
        det = 0
        for j in range(len(A)):
            submatrix = [row[:j] + row[j+1:] for row in A[1:]]
            det += (-1) ** j * A[0][j] * determinant(submatrix)
        return det

def minor(A, i, j):
    return [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]

def cofactor_matrix(A):
    m = len(A)
    n = len(A[0])
    result = [[(-1) ** (i+j) * determinant(minor(A, i, j)) for j in range(n)] for i in range(m)]
    return result

def inverse(A):
    det_A = determinant(A)
    if det_A == 0:
        raise ValueError("Matrix is singular")
    cofactors = cofactor_matrix(A)
    adjugate = transpose(cofactors)
    inv_det_A = 1 / det_A
    result = [[adjugate[i][j] * inv_det_A for j in range(len(A[0]))] for i in range(len(A))]
    return result

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(2, 3)
    if n == 2:
        # Known invariants for determinant of 2x2 matrix
        det_inv_1 = lambda x, y: x[0][0] * x[1][1] - x[0][1] * x[1][0]
        det_inv_2 = lambda x, y: x[0][0] + x[1][1]
        det_inv_3 = lambda x, y: x[0][1] - x[1][0]

        # Known invariants for permanent of 2x2 matrix
        perm_inv_1 = lambda x, y: x[0][0] * x[1][1] + x[0][1] * x[1][0]
        perm_inv_2 = lambda x, y: x[0][0] + x[1][1]

        det_invariants = [det_inv_1, det_inv_2, det_inv_3]
        perm_invariants = [perm_inv_1, perm_inv_2]
    else:
        # For larger n, we cannot easily compute the exact invariants
        return {
            "metric_name": "Invariant Polynomial Count",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

```

```

det_invariant_count = len(det_invariants)
perm_invariant_count = len(perm_invariants)

return {
    "metric_name": "Invariant Polynomial Count",
    "metric_value": det_invariant_count - perm_invariant_count,
    "instances_tested": 1,
    "conjecture_holds": det_invariant_count < perm_invariant_count,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

_name': 'Invariant Polynomial Count', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': F
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Invariant Polynomial Count', 'metric_value': 1, 'instances_tested': 1, 'conje
RESULT: FALSIFIED counterexample="" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	88635
2	preregistration	ollama_remote	qwen3:8b	0	0	24106
3	novelty	ollama_remote	qwen3:8b	0	0	22665
4	novelty	ollama_remote	qwen3:8b	0	0	21688
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15565
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14821
7	critic	ollama_remote	qwen3:8b	0	0	30127
8	judge	ollama_remote	qwen3:8b	0	0	25478

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 243083 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/934a387d12d9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/934a387d12d9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/934a387d12d9.tar.gz (if generated)